Chapter 9: The 3D Structure Prediction Revolution

Chapter 8 asked what a protein language model's internal representations are worth on their own — zero-shot mutation scoring, no 3D coordinates anywhere in sight. This chapter asks the question those representations were, historically, built toward: given only a sequence, can a model output the actual 3D structure that sequence folds into, at experimental accuracy? Since 2020, the answer has gone from "not usefully" to "often, yes" — the single largest discontinuity in this book's subject matter, and the reason Part III exists as three chapters (9, 10, 11) rather than one. This chapter covers the mechanics of *how* that became possible (Sections 9.1–9.3) and puts real programmatic structure prediction to work end to end (Section 9.4): folding two real proteins, validating one against its real crystal structure, and testing whether the model's own confidence score means what it claims to.

9.1 AlphaFold2 & AlphaFold3 Mechanics

AlphaFold2 (Jumper et al., 2021 — introduced in Chapter 3 §3.2 as a forward reference, and cited by Chapter 8 §8.3 as the architectural contrast to ESM-2′s alignment-free design; both threads converge here) reformulated structure prediction as two coupled problems solved jointly: extracting evolutionary and geometric signal from a multiple sequence alignment (Chapter 8 §8.1′s classical starting point), and using that signal to directly output 3D atomic coordinates, with no intermediate contact-map-then-fold pipeline of the kind that dominated the field before it.

The Evoformer. Given a query sequence, AlphaFold2 first builds an MSA the ordinary way (Chapter 3 §3.1′s tool) and initializes two representations from it: an **MSA representation** (one row per homologous sequence, one column per alignment position) and a **pair representation** (one entry per residue-residue pair of the query, initialized from pairwise sequence features). The Evoformer is a stack of blocks that repeatedly updates both representations and exchanges information between them: row-wise and column-wise self-attention over the MSA representation extract co-evolutionary signal (the same conserved-position and co-evolving-pair patterns named in Chapter 8 §8.1, extracted here explicitly from an alignment rather than learned implicitly from a pretraining corpus); an outer-product-mean operation folds updated MSA information into the pair representation; and a set of **triangular update** operations act on the pair representation alone, updating the edge between residues i and j using information from a third residue k — structured so the three edges of every residue triple (i, j, k) are updated consistently with each other, which Jumper et al. (2021) motivate as encoding something close to a geometric triangle inequality directly into the pair representation's learned updates. After 48 such blocks, the pair representation carries a rich, geometrically consistent residue-residue relationship map — not yet 3D coordinates, but the information a 3D structure would need to be consistent with.

Invariant Point Attention (IPA). Converting that pair representation into actual atomic coordinates is the **structure module**'s job, and IPA is its core operation. Every residue carries a local reference frame $T_i = (R_i, \mathbf{t}_i) \in SE(3)$ — a rotation and translation placing that residue's idealized local backbone geometry into the structure's global coordinate frame, initialized identically for every residue (a "black hole" starting guess with no structure yet) and iteratively refined block by block. IPA computes attention not just over the usual scalar query/key features, but also over **3D query and key points** — vectors predicted in each residue's own local frame, mapped into the

shared global frame via each residue's current T_i before their pairwise distances are computed. An attention logit between residues i and j combines three terms: an ordinary scalar dot product, a bias read directly from the Evoformer's pair representation (the geometric information computed in the previous paragraph, now consumed rather than produced), and a penalty growing with the distance between residue i's and j's 3D query/key points once both are placed in the global frame. Because every point is generated in a local frame and only ever compared *after* applying the residues' own current rigid-body transforms, IPA's output is provably invariant to any global rotation or translation applied to the whole structure — the network never has to learn "north" is meaningless, that invariance is built into the operation itself. Each IPA block updates every T_i a little further toward self-consistency; after enough blocks, applying the final frames to a fixed set of idealized atomic coordinates per residue produces the predicted structure.

AlphaFold3. Abramson et al. (2024) replace the structure module described above with a **diffusion model**: rather than iteratively refining rigid per-residue frames, AlphaFold3 starts from atomic coordinates corrupted with Gaussian noise and learns to iteratively denoise them back to a physically valid structure, conditioned on a simplified Evoformer-style trunk (renamed the "Pairformer," with the MSA processing trimmed down). The practical payoff of that architectural change is generality: because diffusion over raw atom coordinates does not depend on the twenty-amino-acid-specific idealized backbone geometry IPA's frame-and-fixed-atoms construction assumes, the same model architecture natively handles nucleic acids, ligands, ions, and covalent modifications as first-class inputs, alongside protein chains — Section 9.3 returns to exactly this capability. AlphaFold3 is discussed here for architectural contrast only; nothing in this chapter's hands-on project runs it.

9.2 ESMFold & RoseTTAFold

Both models below solve the same coordinate-prediction problem as Section 9.1, with one shared, consequential difference: neither performs an MSA search at inference time.

RoseTTAFold (Baek et al., 2021) predates the specific architecture above but shares its core insight of coupling multiple representations: a **three-track network** that processes 1D (sequence), 2D (pairwise distance/orientation), and 3D (explicit atomic coordinate) information simultaneously, with information exchanged between all three tracks at every block, rather than a strictly sequential embed-then-fold pipeline. Like AlphaFold2, its original form still consumes an MSA, so it is introduced here for architectural contrast — a second, independently developed confirmation that coupled multi-representation processing is the right shape for this problem — rather than run in this chapter's hands-on project; Section 9.3 returns to a very different, alignment-free descendant of this same codebase.

ESMFold (Lin et al., 2023, introduced in Chapter 8 §8.2) is this chapter's hands-on model, and its defining move is to delete the MSA step entirely. Recall Chapter 8′s own framing: ESM-2′s masked-language-model pretraining, run once at enormous scale across the observed protein universe, causes evolutionary and structural information to emerge as a side effect and get baked directly into the model's learned weights — no per-query alignment required, because the alignment-derived signal a structure predictor would otherwise need is already implicit in ESM-2′s pretrained

representations. ESMFold exploits this directly: it attaches a structure-prediction "folding trunk" and IPA-based structure module (architecturally descended from AlphaFold2′s, per Section 9.1) directly on top of a frozen-then-fine-tuned ESM-2 representation of a *single* query sequence, with no MSA search step at inference time at all. Lin et al. (2023) report this trades some raw accuracy against AlphaFold2 on hard cases (correlated, as expected, with how much a target actually benefits from cross-sequence evolutionary information an MSA would supply) for roughly 60x faster inference — the practical property this chapter's hands-on project exploits directly, predicting real structures in a couple of seconds each rather than the minutes-to-hours an MSA search against a full sequence database would otherwise cost. AlphaFold2′s own alignment-free variant (a single-sequence input to the ordinary Evoformer, with no MSA) has also been reported to be viable in restricted settings, but ESMFold — purpose-built for this mode rather than adapted to it — is this chapter's more direct example.

9.3 Macromolecular Complexes

Every prediction discussed so far is a single chain, folding on its own. Most of biology, and most drug targets, are not: multi-subunit complexes, and single proteins in complex with the small molecules or other biomolecules the rest of this book concerns itself with.

AlphaFold-Multimer (Evans et al., 2021) extends AlphaFold2 to multiple chains with two concrete changes: the MSA search is run per chain and then the per-chain MSAs and pair representations are concatenated (with a modified positional-encoding scheme that resets per chain, so the model can tell "residue 12 of chain A" apart from "residue 12 of chain B"), and training incorporates a loss term specifically rewarding correct inter-chain contacts, not just correct per-chain folds. As of this writing, Evans et al. (2021) remains a bioRxiv preprint with no separate peer-reviewed-venue publication, verified directly against Crossref rather than assumed; its findings are nonetheless the field's standard reference point for multimeric AlphaFold2-family prediction.

Protein-ligand co-structure prediction is a different extension of the same underlying problem: given a protein sequence *and* a small-molecule graph (Chapter 2′s representation), predict their bound complex directly, rather than predicting the protein structure alone and docking a ligand into it afterward (Chapter 11′s subject). AlphaFold3′s diffusion-based structure module (Section 9.1) was built specifically to make this tractable — because it denoises raw atomic coordinates rather than applying per-residue rigid backbone frames, a bound ligand's atoms are just more atoms to denoise, subject to the same process as the protein chain, rather than a fundamentally different kind of input needing a separate mechanism. RoseTTAFold All-Atom (Krishna et al., 2024) generalizes RoseTTAFold's three-track architecture (Section 9.2) in the same direction, for the same reason: broadening the "atomic" track to represent arbitrary chemistry, not just the twenty standard amino acids, so proteins, nucleic acids, and small molecules share one network rather than requiring separate specialized models bolted together. Both are introduced here as the architectural bridge between this chapter's single-chain structure prediction and Chapter 11′s docking methods, and are not run in this chapter's hands-on project.

9.4 Hands-on Project: ESMFold Structure Prediction & pLDDT Validation

The project code lives in this chapter's folder (`ch09_structure_prediction/`). It puts two real, contrasting protein sequences through ESMFold, run programmatically rather than by hand, and asks whether the model's self-reported confidence — **pLDDT** (predicted Local Distance Difference Test, a per-residue score in [0, 100] in the original papers, estimating how well the predicted local backbone geometry would agree with a hypothetical true structure) — means what it claims to, against two different, real kinds of ground truth.

A feasibility decision, made explicit

The most direct way to "run ESMFold programmatically" is to load `facebook/esmfold_v1` locally (available via Hugging Face `transformers`' `EsmForProteinFolding`, confirmed importable in this project's environment). That checkpoint is real, but large: its weights file is approximately 8.4 GB, because `esmfold_v1` is built on ESM-2's 3-billion-parameter backbone (Chapter 8 §8.2's scale sweep, at its largest tested size), not the 8-150M-parameter checkpoints Chapter 8's project used. This project's development machine has 16 GB of total RAM with roughly 7 GB free under normal load and no CUDA GPU; loading an 8.4 GB checkpoint (before accounting for activation memory during inference) risks exhausting available memory outright, and CPU inference through a 3B-parameter transformer trunk for even one query sequence would likely take on the order of tens of minutes per prediction. Rather than attempt that and risk an unreliable, possibly unreproducible run, this project uses the real, GPU-hosted `esmfold_v1` model via Meta's public **ESM Metagenomic Atlas** inference API — the same weights, called over HTTPS, documented as the Atlas's live folding endpoint in Meta's own `esm` GitHub repository. This is a genuine instance of "running ESMFold programmatically": a real Python `requests` call, real model inference on Meta's infrastructure, and real 3D coordinates back, without a local download this project's hardware cannot safely absorb — and, being free-tier-friendly with no GPU requirement, closer in spirit to the "free-tier Google Colab" target environment this book's tech stack commits to than a local 3B-parameter model would be.

That choice has a real, measured cost, reported here rather than glossed over. The API enforces a hard request timeout; repeated, directly measured trials (`curl`, timed) found it reliably folds real sequences up to at least 140 residues in 1-3 seconds, but two different 293-residue and disordered-91-residue real sequences tested during this project's development both timed out at a consistent ~30 seconds, regardless of retry. This determined which real sequences the experiment below uses — both well under the length where timeouts were observed — rather than the two-chain EGFR kinase domain (Chapter 1/3's 293-residue running example) originally considered for this section.

Real data: one ordered protein, one disordered protein

Ubiquitin (UniProt P0CG47, the 76-residue repeat unit of human polyubiquitin-B) is a compact, single-domain, extremely well-characterized protein with an ultra-high-resolution (1.8 Å) crystal structure, PDB `1UBQ` (Vijay-Kumar et al., 1987) — bundled directly in this project (`data/1UBQ.pdb`) the same way Chapter 3 bundles `1M17.pdb`. Its real, experimentally determined structure is the ground truth Section 9.4's first evaluation validates against.

Alpha-synuclein (UniProt P37840, 140 residues, full canonical sequence) is the contrasting case: DisProt (Aspromonte et al., 2024), the manually curated database of experimentally characterized intrinsically disordered regions, annotates its *entire* length as disordered (entry DP00070), independently supported by NMR, far-UV circular dichroism, and SDS-PAGE evidence — not a computational prediction, a directly observed experimental property. Unlike ubiquitin, there is no single native fold for a predicted structure to be "right" or "wrong" against; the honest test here is whether ESMFold's confidence score correctly reports low confidence rather than confidently hallucinating a fold that does not exist.

```
UBIQUITIN_SEQUENCE = (
"MQIFVKTLTGKTITLEVEPSDTIENVKAKIQDKEGIPPDQQRLIFAGKQLEDGRTLSDYNIQKESTLHLVLRLRGG"
)  # UniProt P0CG47, 76 aa
ALPHA_SYNUCLEIN_SEQUENCE = (
"MDVFMKGLSKAKEGVVAAAEKTKQGVAEAAGKTKEGVLYVGSKTKEGVVHGVATVAEKTKEQVTNVGGAVVTGVTA"
    "VAQKTVEGAGSIAAATGFVKKDQLGKNEEGAPQEGILEDMPVDPDNEAYEMPSEEGYQDYEPEA"
)  # UniProt P37840, 140 aa
```

Predicting, and reading pLDDT correctly

`esmfold_structure_prediction.py`'s `fold_sequence` POSTs a raw sequence to the Atlas API and returns the predicted structure as PDB text. One real, verified detail matters for everything downstream: this API reports pLDDT already normalized to $[0, 1]$, not the $[0, 100]$ scale the original papers use — confirmed directly by inspecting the returned B-factor column's value range across every sequence used in this project, not assumed from the papers' convention. A second, equally verified detail: pLDDT here varies *per atom* within a residue's B-factor column, not as one value repeated across all of a residue's atoms (AlphaFold's own reference implementation's convention). `per_residue_plddt` accounts for both, averaging each residue's per-atom values into one summary confidence per residue.

```
def per_residue_plddt(pdb_text: str) -> list[ResidueConfidence]:
    parser = PDBParser(QUIET=True)
    structure = parser.get_structure("prediction", StringIO(pdb_text))
    chain = structure[0]["A"]
    result = []
    for residue in chain:
        if residue.id[0] != " ":
            continue
        atom_plddts = [atom.get_bfactor() for atom in residue]
        result.append(ResidueConfidence(residue_number=residue.id[1],
plddt=float(np.mean(atom_plddts))))
    return result
```

Evaluating against real ground truth

`compute_ca_accuracy` superimposes the predicted ubiquitin structure onto the real `1UBQ` crystal structure using BioPython's `Superimposer`, matching C-alpha atoms 1:1 by residue number (both the prediction and the crystal structure use identical 1-76 numbering, confirmed directly rather than assumed) to obtain the global RMSD, and each residue's individual post-alignment deviation:

```
sup = Superimposer()
sup.set_atoms(ref_ca, pred_ca)  # fit predicted onto reference
sup.apply(pred_ca)
per_residue_deviation = {
    resnum: float(np.linalg.norm(pred_atom.coord - ref_atom.coord))
    for resnum, pred_atom, ref_atom in zip(shared_resnums, pred_ca, ref_ca)
}
```

`evaluate_confidence_vs_accuracy` then computes the Spearman correlation between each residue's pLDDT and its post-alignment deviation from the real structure; `evaluate_disorder_signal` runs a Mann-Whitney U test comparing ubiquitin's and alpha-synuclein's full per-residue pLDDT distributions.

Running it and reading the results

```
python esmfold_structure_prediction.py --use-cached-raw
```

Run against the real ESMFold predictions (live API results, verified bit-identical to the bundled offline fixture before either number below was taken from either source):

Comparison	Result
Ubiquitin vs. real crystal structure (`1UBQ`)	Global C-alpha RMSD = **0.827 Å** (n=76 residues)
pLDDT vs. per-residue C-alpha deviation	Spearman ρ = **-0.530** (p = 8.68e-07)
Mean pLDDT: ubiquitin (ordered)	**0.858**
Mean pLDDT: alpha-synuclein (disordered)	**0.315**
Mann-Whitney U (ordered > disordered)	U = 10629.0, p = 5.14e-34

Three things are worth reading precisely. First, a sub-angstrom global C-alpha RMSD against a real, independently determined 1.8 Å crystal structure, from a single sequence with zero MSA search and zero knowledge of that structure, is a genuinely strong result — not a cherry-picked one; ubiquitin is exactly the kind of compact, abundant-in-training-data, single-domain fold ESMFold performs best on, and Section 9.2′s "60x faster, some accuracy cost" framing describes *harder* targets than this one, not this one. Second, the correlation between pLDDT and real structural error is negative and highly significant ($\rho = -0.530$, $p < 10^{-6}$) — confidence really does track accuracy here, more cleanly than Chapter 8′s zero-shot mutation scores tracked experimental fitness ($\rho \approx 0.2$–0.3 at best), consistent with pLDDT being trained with a direct, structure-specific supervision signal rather than repurposed from an unrelated pretraining objective the way Chapter 8′s masked-marginal scores were. Third, the pLDDT gap between the ordered and disordered protein is large (0.858 vs. 0.315) and the Mann-Whitney test is unambiguous ($p \approx 5 \times 10^{-34}$, $n = 76$ vs. $n = 140$ residues): ESMFold does not confidently hallucinate a fold for alpha-synuclein, it reports low confidence across essentially its entire length — the correct, honest behavior for a protein DisProt independently confirms has no single native fold, and a direct,

small-scale replication of the pLDDT-disorder association Wilson et al. (2022) report at larger scale.

Why not PAE?

The outline for this section names both pLDDT and **PAE** (Predicted Aligned Error — a pairwise, not per-residue, confidence estimate: for every residue pair (i, j), the expected position error of residue j if the structure were instead aligned on residue i's local frame). Two real reasons, not one, kept it out of this section's actual computation. Practically, the lightweight public API used here — chosen over a local 8.4 GB, 3B-parameter checkpoint for the feasibility reasons given above — returns only a PDB structure with pLDDT in the B-factor column; obtaining PAE requires the local Python inference path (`model.infer(...)`'s full output dictionary), which was the specific option this project's hardware ruled out. Scientifically, PAE is most informative exactly where Section 9.3′s multimeric and multi-domain cases apply — assessing how confidently two chains, or two loosely connected domains, are placed *relative to each other* — a question that does not meaningfully arise for a single small ordered domain like ubiquitin evaluated here, whose one compact fold makes intra-chain relative placement unambiguous once pLDDT is already high throughout. Section 9.4 stays scoped to what it can validate rigorously against real ground truth; PAE's natural evaluation belongs with Chapter 10′s multi-part designed structures.

Reproducibility

Dependencies are version-floored (`biopython>=1.81`, `requests>=2.28`, `scipy>=1.10` in `requirements.txt`, validated against biopython 1.88, requests 2.34.2, and scipy 1.18.0 on Python 3.12). `data/ubiquitin_esmfold_prediction.pdb` and `data/alpha_synuclein_esmfold_prediction.pdb` bundle the real API responses (fetched 2026-08-20), so `--use-cached-raw` runs the full pipeline offline and deterministically; `data/1UBQ.pdb` bundles the real RCSB crystal structure. Both were verified directly: a fresh live call to the API for each sequence, made independently of the bundled files, produced byte-identical PDB text, confirming ESMFold's inference is deterministic for a fixed sequence and that the bundled fixtures are not stale. The 14-test suite in `tests/test_esmfold_structure_prediction.py` runs against these same real fixtures (parsing, superposition correctness — including a self-alignment sanity check that must produce ~0 RMSD — and both evaluation functions' statistical sign and shape), with exactly one test calling the live API directly. `pip install -r requirements-dev.txt && pytest` reproduces all 14 results.

Limitations and what comes next

This section validates exactly two real proteins in detail, chosen for what they could rigorously demonstrate (one against real crystallographic ground truth, one against a real experimental disorder annotation) within this project's measured API constraints — not a general accuracy benchmark across many folds, which would need the kind of large curated test set Section 9.1′s cited papers themselves report against. It evaluates monomeric, single-chain structure prediction only; Section 9.3′s multimeric and co-structure extensions are covered as theory, not run here. And, as detailed above, it uses pLDDT exclusively, not PAE, for reasons that are architectural-scope decisions as much as hardware ones. Chapter 10 picks up directly from here: given a reliable

way to predict and confidence-score a structure from sequence (this chapter), how does one instead *design* a novel sequence, or even a novel backbone, meant to fold into a desired structure or bind a desired target in the first place?

A note on Google Colab

`requests`, `numpy`, and `scipy` are preinstalled on Colab's default runtime; only `biopython` needs `! pip install biopython`. No GPU is required — this project calls a hosted API rather than running any model locally, so it runs identically on Colab's free CPU-only tier.

References

- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Zidek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S. A. A., Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., … Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589. https://doi.org/10.1038/s41586-021-03819-2
- Abramson, J., Adler, J., Dunger, J., Evans, R., Green, T., Pritzel, A., Ronneberger, O., Willmore, L., Ballard, A. J., Bambrick, J., Bodenstein, S. W., Evans, D. A., Hung, C.-C., O'Neill, M., Reiman, D., Tunyasuvunakool, K., Wu, Z., Zemgulyte, A., Arvaniti, E., … Jumper, J. M. (2024). Accurate structure prediction of biomolecular interactions with AlphaFold3. *Nature*, 630(8016), 493-500. https://doi.org/10.1038/s41586-024-07487-w
- Baek, M., DiMaio, F., Anishchenko, I., Dauparas, J., Ovchinnikov, S., Lee, G. R., Wang, J., Cong, Q., Kinch, L. N., Schaeffer, R. D., Millan, C., Park, H., Adams, C., Glassman, C. R., DeGiovanni, A., Pereira, J. H., Rodrigues, A. V., van Dijk, A. A., … Baker, D. (2021). Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557), 871-876. https://doi.org/10.1126/science.abj8754
- Krishna, R., Wang, J., Ahern, W., Sturmfels, P., Venkatesh, P., Kalvet, I., Lee, G. R., Morey-Burrows, F. S., Anishchenko, I., Humphreys, I. R., McHugh, R., Vafeados, D., Li, X., Sutherland, G. A., Hitchcock, A., Hutchinson, C. N., Kang, A., Endelman, B., … Baker, D. (2024). Generalized biomolecular modeling and design with RoseTTAFold All-Atom. *Science*, 384(6693), eadl2528. https://doi.org/10.1126/science.adl2528
- Evans, R., O'Neill, M., Pritzel, A., Antropova, N., Senior, A., Green, T., Zidek, A., Bates, R., Blackwell, S., Yim, J., Ronneberger, O., Bodenstein, S., Zielinski, M., Bridgland, A., Potapenko, A., Cowie, A., Tunyasuvunakool, K., Jain, R., … Hassabis, D. (2021). Protein complex prediction with AlphaFold-Multimer. *bioRxiv*. https://doi.org/10.1101/2021.10.04.463034 (no separate peer-reviewed-venue DOI was found for this paper as of this writing, verified directly against Crossref rather than assumed; the bioRxiv preprint DOI is cited instead, its status disclosed here rather than implied otherwise).
- Wilson, C. J., Choy, W.-Y., & Karttunen, M. (2022). AlphaFold2: A Role for Disordered Protein/Region Prediction? *International Journal of Molecular Sciences*, 23(9), 4591. https://doi.org/10.3390/ijms23094591
- Aspromonte, M. C., Nugnes, M. V., Quaglia, F., Bouharoua, A., DisProt Consortium, & Tosatto, S. C. E. (2024). DisProt in 2024: improving function annotation of intrinsically disordered proteins. *Nucleic Acids Research*, 52(D1), D434-D441. https://doi.org/10.1093/nar/gkad928

- Vijay-Kumar, S., Bugg, C. E., & Cook, W. J. (1987). Structure of ubiquitin refined at 1.8 Å resolution. *Journal of Molecular Biology*, 194(3), 531-544. https://doi.org/10.1016/0022-2836(87)90679-6

See Chapter 1′s references for the wwPDB consortium (2019) PDB/mmCIF database citation, reused here rather than re-listed. See Chapter 8′s references for Lin et al. (2023, ESMFold/ESM-2), reused here as this chapter's primary hands-on model. RDKit, DeepChem, XGBoost, and PyTorch Geometric, used throughout Chapters 1-7, play no direct role in this chapter's hands-on code; ESMFold predictions were obtained from Meta's public ESM Metagenomic Atlas API (`api.esmatlas.com`), documented in the `facebookresearch/esm` GitHub repository, and the real UniProt sequences and DisProt disorder annotations used here were fetched directly from `rest.uniprot.org` and `disprot.org` rather than transcribed from a secondary source.

All RMSD, correlation, and p-values cited in Section 9.4 were computed directly by running `esmfold_structure_prediction.py` against the live ESM Metagenomic Atlas API on 2026-08-20, not taken from a secondary source, and independently reproduced against the bundled offline fixtures — see `data/` and `results/esmfold_structure_results.json` to reproduce.